

CONTENT PROVIDER

Prof.K.Sathiyamurthy, DCSE,PTU

Content Provider Fundamentals

Building Blocks of Android

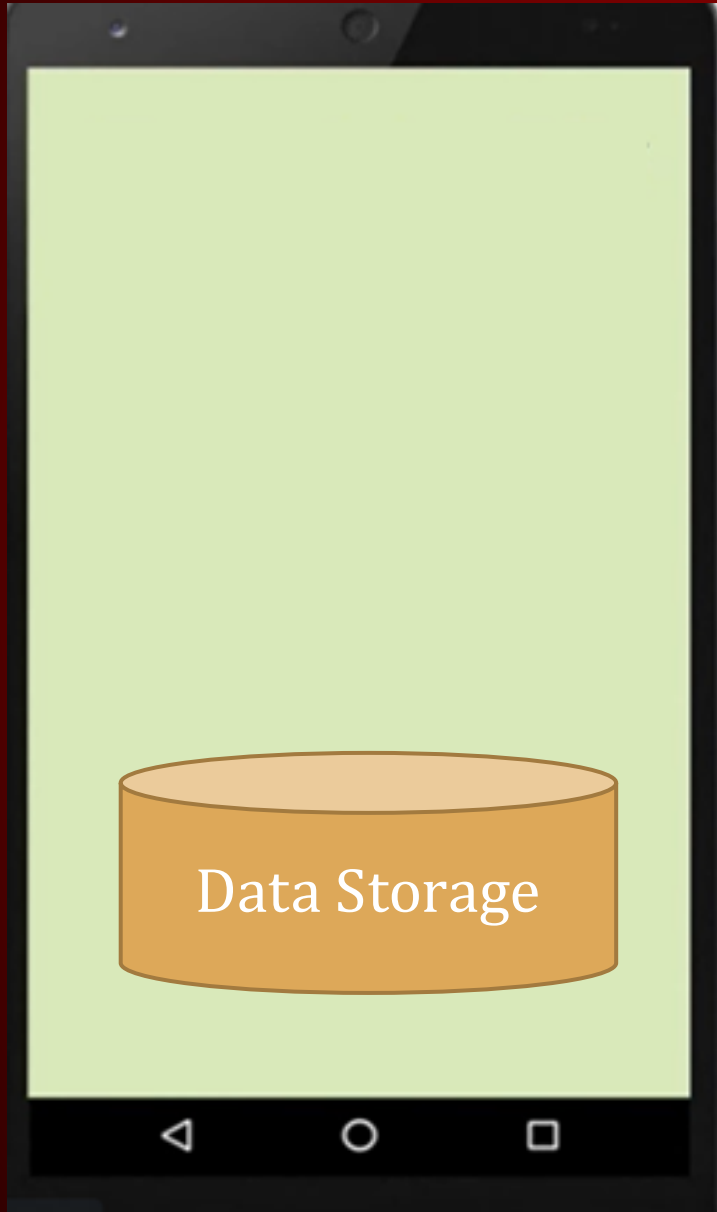
Activity

Service

BroadcastReceiver

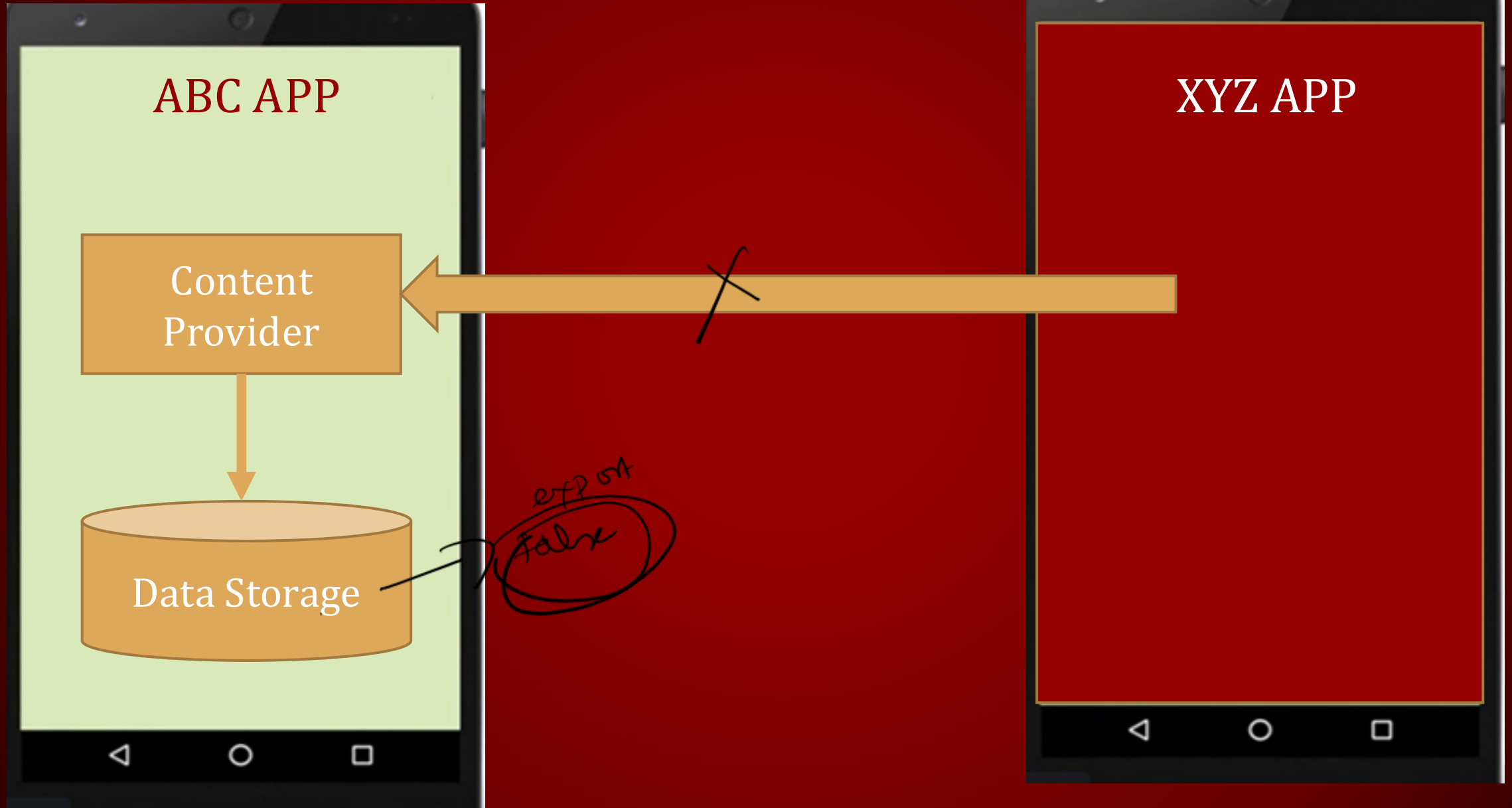
ContentProvider

Content Provider

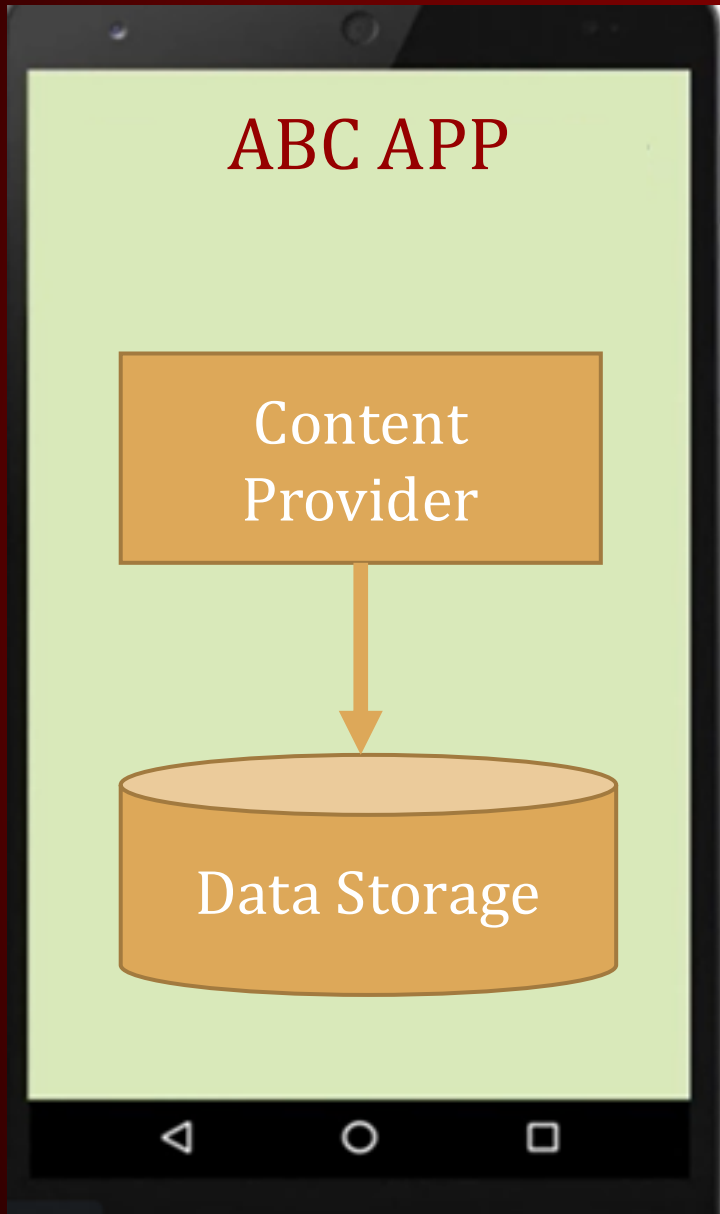


- ❑ Data Storage – can contain file, SQLite DB, audio, video, photos...
- ❑ DS is private to our own applications, no outside application is allowed to access this data storage
- ❑ Other applications are not allowed to access the DS
- ❑ CP has to be used for other applications to access the DS inside our application
- ❑ Provides content of our application to some other application

Content Provider



Content Provider –Data Storage Options



Content Provider Offers data storage in two ways

- File Data
 - Photos, videos, Audio Files, text, JSON
- Structured Data
 - SQLite database

Content Provider

- ❑ Android Component
 - ❑ Similar to activity, service or Broadcast Receiver
- ❑ Manages access to central repository of data
 - ❑ File Data-Audio,video,photo, text files
 - ❑ Structured data-SQLite DB, Arrays..
- ❑ Securely exposes private data outside app
 - ❑ Maintains app security

Need of Content Provider

- ❑ To share complex data or files to some other applications
- ❑ To securely share your private data to other apps
- ❑ Using search framework to implement custom search suggestion
- ❑ To present list of data using CursorLoader, CursorAdapter

Steps to create Content Provider

- ❑ Define Contract class
 - ❑ Contains DB configuration constants
 - ❑ Table name, Column names
- ❑ Define Database Helper class
 - ❑ It helps to create database
 - ❑ Also manages database versions
- ❑ Performs DB operations
 - ❑ Insert
 - ❑ Query:print all contents of database
 - ❑ Update
 - ❑ Delete

Content Provider — Contract Class (Code)

- ❑ Holds the table name, column names, authority, and content URI — nothing else
- ❑ Every other class agrees on these names instead of retyping raw strings

```
public static final String AUTHORITY =  
    "cse.pec.sathiya.cpdemo1.data.NationProvider";  
  
public static final Uri CONTENT_URI =  
    Uri.parse("content://" + AUTHORITY + "/countries");  
  
public static final String TABLE_NAME = "countries";  
public static final String COLUMN_NAME = "name";  
public static final String COLUMN_CAPITAL = "capital";
```

Content Provider — Database Helper (Code)

- ❑ Same SQLiteOpenHelper pattern as StudentDbHelper — creates the table once, migrates it on version bumps
- ❑ Only NationProvider is ever allowed to call this class

```
public void onCreate(SQLiteDatabase db) {
    db.execSQL("CREATE TABLE countries (" +
        "_id INTEGER PRIMARY KEY AUTOINCREMENT, " +
        "name TEXT NOT NULL, capital TEXT NOT NULL)");
}

public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
    db.execSQL("DROP TABLE IF EXISTS countries");
    onCreate(db);
}
```

Content Provider-Query Operations

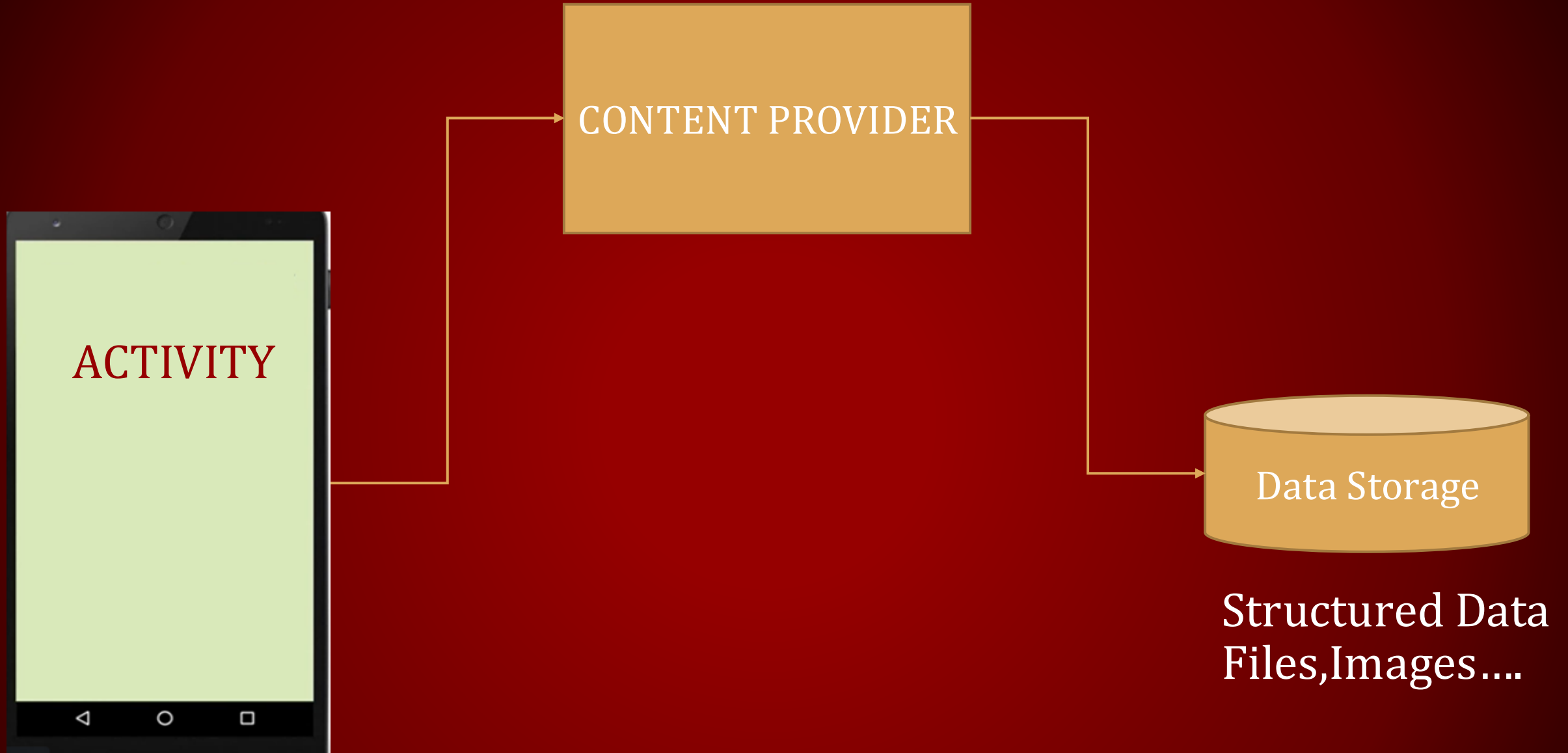
- ❑ Projection
 - Desired columns
- ❑ Selection: Where clause
 - To filter our rows
- ❑ Selection Args
 - Helps to filter our specific results
- ❑ Sort order
 - Ascending or Descending

Content Provider — query() Implementation (Code)

- ❑ UriMatcher decides whether the URI means the whole table or a single row
- ❑ Same projection / selection / selectionArgs / sortOrder from the previous slide, now in real code

```
switch (sUriMatcher.match(uri)) {  
    case COUNTRIES:  
        return db.query(TABLE_NAME, projection, selection,  
                        selectionArgs, null, null, sortOrder);  
  
    case COUNTRY_ID:  
        selection = "_id=?";  
        selectionArgs = new String[]{  
            String.valueOf(ContentUris.parseId(uri))};  
        return db.query(TABLE_NAME, projection, selection,  
                        selectionArgs, null, null, sortOrder);  
}
```

IMPLEMENTING CONTENT PROVIDER



Steps to Implement Content Provider

1. Design raw storage for your data
 1. File Data:Photos,audio,video stored in apps private storage
 2. Structured data:SQLite DB, arrays etc
2. Create subclass of contentProvider
 1. Override the abstract methods
3. Define Authority String, its content URIs and column names
4. Declare contentProvider inside manifest file

Declaring the Content Provider (Code)

- ❑ Registered in AndroidManifest.xml, inside <application> — not in Java code
- ❑ authorities must exactly match the AUTHORITY constant in the Contract class

```
<provider
    android:name=".data.NationProvider"
    android:authorities=
        "cse.pec.sathiya.cpdemo1.data.NationProvider"
    android:exported="false" />
```

Content Provider Methods

OnCreate

- Called when application starts
- Use it to initialize ContentProvider

Query

- To retrieve results from database

Insert

- insert data into database

Delete

- Delete records

Update

- update records

getType

- Handle requests for the MIME type of the data for the given URI

Content Provider — onCreate() & insert() (Code)

- ❑ onCreate() runs once, when the provider is first needed
- ❑ insert() adds a row, then returns the URI of the row just created

```
public boolean onCreate() {
    dbHelper = new NationDbHelper(getContext());
    return true;
}

public Uri insert(Uri uri, ContentValues values) {
    long id = db.insert(TABLE_NAME, null, values);
    getContext().getContentResolver()
        .notifyChange(uri, null);
    return ContentUris.withAppendedId(CONTENT_URI, id);
}
```

Content Provider — update(), delete() & getType() (Code)

- ❑ update() and delete() match rows exactly like query() did — same switch, same WHERE clause
- ❑ delete()'s body is identical to update()'s — it just calls db.delete() instead of db.update()

```
public int update(Uri uri, ContentValues values,
    String selection, String[] args) {
    int rows = db.update(TABLE_NAME, values, selection, args);
    if (rows != 0)
        getContext().getContentResolver().notifyChange(uri, null);
    return rows;
}

public String getType(Uri uri) {
    return sUriMatcher.match(uri) == COUNTRIES
        ? CONTENT_LIST_TYPE : CONTENT_ITEM_TYPE;
}
```

Content URIs

Content://<authority>/path_name



Content://<package_name>contentprovider/path_name



Content://cse.pec.sathiya.cpdemo1.data.NationProvider/countries

<authority> has to be unique

Use app's package Name with Class name

Path_name:helps client app to reach our database table or file

Content URI Patterns

A content URI pattern uses wildcard characters to match URI:

1. *: Matches a string of any valid characters Example: "name", "54", "3", "hello"
2. #: Matches a string of numeric characters Example: "54", "3", "27"

Content://com.myapp.provider/*

Content://com.myapp.provider/table1 ✓

Content://com.myapp.provider/table1/mydata ✓

Content://com.myapp.provider/table2 ✓

Content://com.myapp.provider/7 ✓

Content://com.myapp.provider/table1/*

Content://com.myapp.provider/table1/mydata ✓

Content://com.myapp.provider/table2 ✗

Content://com.myapp.provider/table1/5 ✓

Content://com.myapp.provider/table1/#

Content://com.myapp.provider/table1/4

Content://com.myapp.provider/table1/hello ✗

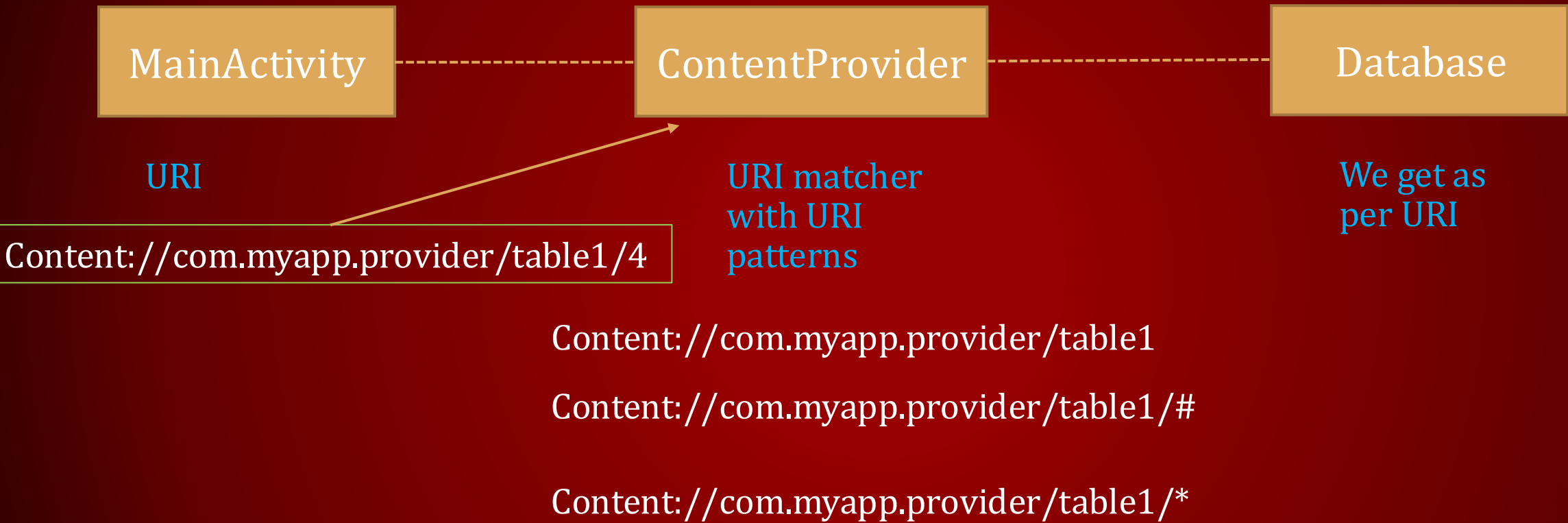
Content://com.myapp.provider/table1/mydata ✗

UriMatcher Setup (Code)

- ❑ Built once, in a static block, when the provider class loads
- ❑ Turns a URI shape into a small int code so query()/insert()/update()/delete() can switch on it

```
private static final int COUNTRIES = 100;
private static final int COUNTRY_ID = 101;

private static final UriMatcher sUriMatcher =
    new UriMatcher(UriMatcher.NO_MATCH);
static {
    sUriMatcher.addURI(AUTHORITY, "countries", COUNTRIES);
    sUriMatcher.addURI(AUTHORITY, "countries/#", COUNTRY_ID);
}
```



Using the Content Provider — Client Code

- ❑ The Activity never touches SQLiteDatabase — only ContentResolver
- ❑ Same four operations as direct SQLite access, one line each

```
getContentResolver().insert(CONTENT_URI, values);
```

```
getContentResolver().query(CONTENT_URI, projection,  
    null, null, sortOrder);
```

```
getContentResolver().update(itemUri, values, null, null);
```

```
getContentResolver().delete(itemUri, null, null);
```